



深圳職業技術大學

SHENZHEN POLYTECHNIC UNIVERSITY

《深度学习进阶》

课程设计报告

项目名称：多风格图片动漫化系统

学 院：人工智能学院

班 级：23 智能 2 班

指导老师：任晋

学生姓名：钟文清

深度学习进阶课程设计报告

多风格图像卡通化系统设计报告

1. 引言 (Introduction)

1.1 课程设计背景

数字图像风格迁移 (Image Style Transfer) 是计算机视觉领域的一个重要分支，其目标是在保持原图像内容结构 (Content) 的同时，将其纹理和色彩转换为特定的艺术风格 (Style)。传统的非真实感绘制 (NPR) 技术通常泛化能力有限，而基于生成对抗网络 (GAN) 的深度学习方法已成为该领域的主流。

本次课程设计旨在构建一个**集成化、多风格**的图像卡通化系统。我们不仅复现了经典的日系动漫风格（如宫崎骏、新海诚），还创新性地引入了欧美 3D 动画风格 (Arcane)，实现了一个横跨东西方审美差异的综合性图像处理平台。

1.2 任务目标

- 多模型集成**：在同一系统中兼容 Pytorch 原生模型 (AnimeGAN2) 和 TorchScript JIT 模型 (ArcaneGAN)。
- 动态风格切换**：实现“热插拔”式的权重加载机制，允许用户在运行时自由切换画风。
- 混合精度适配**：解决不同预训练模型在数据类型 (FP16/FP32) 上的不兼容问题，增强系统的鲁棒性。
- 工程化实现**：基于 Gradio 构建 Web UI，支持图片 / 视频处理、人脸自动裁剪及后处理优化。

2. 系统架构与算法原理 (Architecture & Algorithms)

2.1 系统运行流程图

本系统采用了**双引擎推理架构**，根据用户选择的风格动态路由到不同的处理管线。


```
1 graph TD
2     A[启动 Web UI] --> B{用户选择风格};
3     B -- 日系风格 (Hayao/Shinkai) --> C[引擎 I: AnimeGAN2];
4     B -- 美系风格 (Arcane) --> D[引擎 II: ArcaneGAN (JIT)];
5
6     subgraph 引擎 I (AnimeGAN2)
7         C1[加载 Pytorch State_Dict] --> C2[预处理: Resize & [-1,1] 归一化];
8         C2 --> C3[推理: Generator(x)];
9         C3 --> C4[反归一化: x/2 + 0.5];
10    end
11
12    subgraph 引擎 II (ArcaneGAN)
13        D1[加载 TorchScript .jit 文件] --> D2[预处理: ImageNet 归一化];
14        D2 --> D3[混合精度适配 (FP16/32 自动检测)];
15        D3 --> D4[推理: JIT_Model(x)];
16        D4 --> D5[反归一化];
17    end
18
19    C4 --> E[后处理模块];
20    D5 --> E;
21
22    E --> F{开启平滑?};
23    F -- 是 --> G[高斯滤波 (Gaussian Blur)];
24    F -- 否 --> H[输出结果];
25    G --> H;
```

2.2 核心模型原理

1. AnimeGAN2 (轻量化 GAN)

- **结构**: 采用了反向残差块 (Inverted Residual Blocks) 和 深度可分离卷积, 显著降低了参数量。
- **课内知识运用**: 这验证了我们在课堂上学到的 CNN 优化理论——通过减少通道冗余计算 (Pointwise + Depthwise) 来换取推理速度。

2. ArcaneGAN (风格化微调)

- **技术路线**: 基于 U-Net 架构或 FastAI 框架进行微调, 专门针对《双城之战》的 3D 油画纹理进行训练。
- **部署形式**: 使用 **TorchScript (JIT)** 格式发布, 这是一种独立于 Python 源码的序列化格式, 允许模型在没有原始类定义的情况下运行, 极大方便了跨项目集成。

3. 核心代码与实现细节 (Implementation Details)

3.1 动态权重加载机制

为了在有限显存下支持多模型，我们设计了按需加载逻辑：

Python

```
1 # 风格配置字典：映射风格名称到下载链接
2 STYLE_URLS = {
3     "Hayao": ".../celeba_distill.pt",
4     "Arcane": ".../ArcaneGANv0.4.jit"
5 }
6
7 def load_style_weights(style_name):
8     # 缓存机制：避免重复下载和加载
9     if style_name in loaded_styles:
10         return loaded_styles[style_name]
11
12     # 分支处理：根据模型类型采用不同加载策略
13     if "Arcane" in style_name:
14         # JIT 模型加载
15         jit_model = torch.jit.load(filename).to(device)
16         loaded_styles[style_name] = {"type": "jit", "model": jit_model}
17     else:
18         # Pytorch 权重加载
19         weights = torch.hub.load_state_dict_from_url(url)
20         model.load_state_dict(weights)
```

3.2 鲁棒的推理管线 (`transform_image`)

此函数是系统的“大脑”，处理了最棘手的数据类型匹配问题。

Python

```
1 # 鲁棒的类型匹配逻辑
2 try:
3     output = jit_model(img_tensor)
4 except RuntimeError as e:
5     if "HalfTensor" in str(e):
6         # 异常捕获：如果模型提示需要半精度(FP16)
7         img_tensor = img_tensor.half() # 自动将输入降级为 FP16
8         output = jit_model(img_tensor)
9     else:
10         raise e
```

- **深度解读：**在实际工程中，预训练模型往往由不同环境训练而来（有的使用混合精度训练以节省显存）。如果直接输入默认的 FP32 张量，会导致 CUDA 核心报错。这段代码展示了**防御性编程**的思想，通过 `try-except` 机制动态适配模型的数据类型要求，确保系统不会崩溃。

3.3 图像处理基础算法

- **高斯平滑：** `apply_gaussian_smoothing` 函数利用卷积核对图像进行低通滤波。
 - **原理：** $\frac{1}{2\pi\sigma^2} e^{-\frac{(x^2+y^2)}{2\sigma^2}}$ 。这有效抑制了 GAN 生成图像中常见的棋盘格伪影（Checkerboard Artifacts），使画面更加柔和。
- **人脸检测：**利用 OpenCV 的级联分类器实现感兴趣区域（ROI）的自动提取。

4. 总结与反思 (Conclusion & Reflection)

4.1 课程设计收获

1. **跨模型集成能力：**成功在同一个 Python 进程中管理了 pytorch 原生模型和 JIT 编译模型，理解了 `state_dict` 与 `ScriptModule` 的区别。
2. **全栈开发体验：**从后端的模型推理，到中间件的业务逻辑（自动裁剪、类型转换），再到前端的 Gradio 交互，打通了 AI 应用落地的全链路。
3. **理论验证：**直观体会到了不同 GAN 损失函数（Loss Function）对生成结果的影响——AnimeGAN 的 Gram Matrix Loss 导致了其保留了强烈的边缘线条，而 ArcaneGAN 则更注重色块的体积感。

4.2 不足与改进

1. **视频帧间抖动**：目前视频处理是逐帧进行的，缺乏时间维度的一致性约束（Temporal Consistency），导致画面可能出现闪烁。
 - **改进**：引入光流法（Optical Flow）对前后帧进行约束和平滑。
2. **显存优化**：同时缓存多个模型会占用显存。
 - **改进**：实现 LRU（最近最少使用）缓存淘汰策略，在显存不足时自动卸载旧模型。

4.3 AI 工具使用总结

在本次开发中，AI 辅助工具（Cursor/ChatGPT）扮演了重要角色：

- **提效**：在寻找模型下载链接、编写 Gradio 样板代码方面，AI 节省了约 50% 的时间。
 - **避坑**：在遇到 `RuntimeError: Input type ... should be the same` 这种棘手的底层错误时，AI 提供的关于 JIT 模型混合精度特性的解释至关重要，帮助我快速定位到了 FP16/FP32 不匹配的根源。
 - **思考**：AI 虽强，但无法替代对系统架构的宏观把控。例如，如何设计“双引擎”架构来兼容不同模型接口，仍需开发者具备扎实的软件工程思维。
-

附录

1. 环境依赖

本项目完全基于课程提供的 `AI_gpu` 环境开发，核心依赖库如下：

- `torch` (PyTorch 深度学习框架)
- `cv2` (OpenCV 图像处理)
- `gradio` (Web UI 界面)
- `numpy` & `PIL` (矩阵与图像基础库)

无需安装任何额外第三方包即可运行。

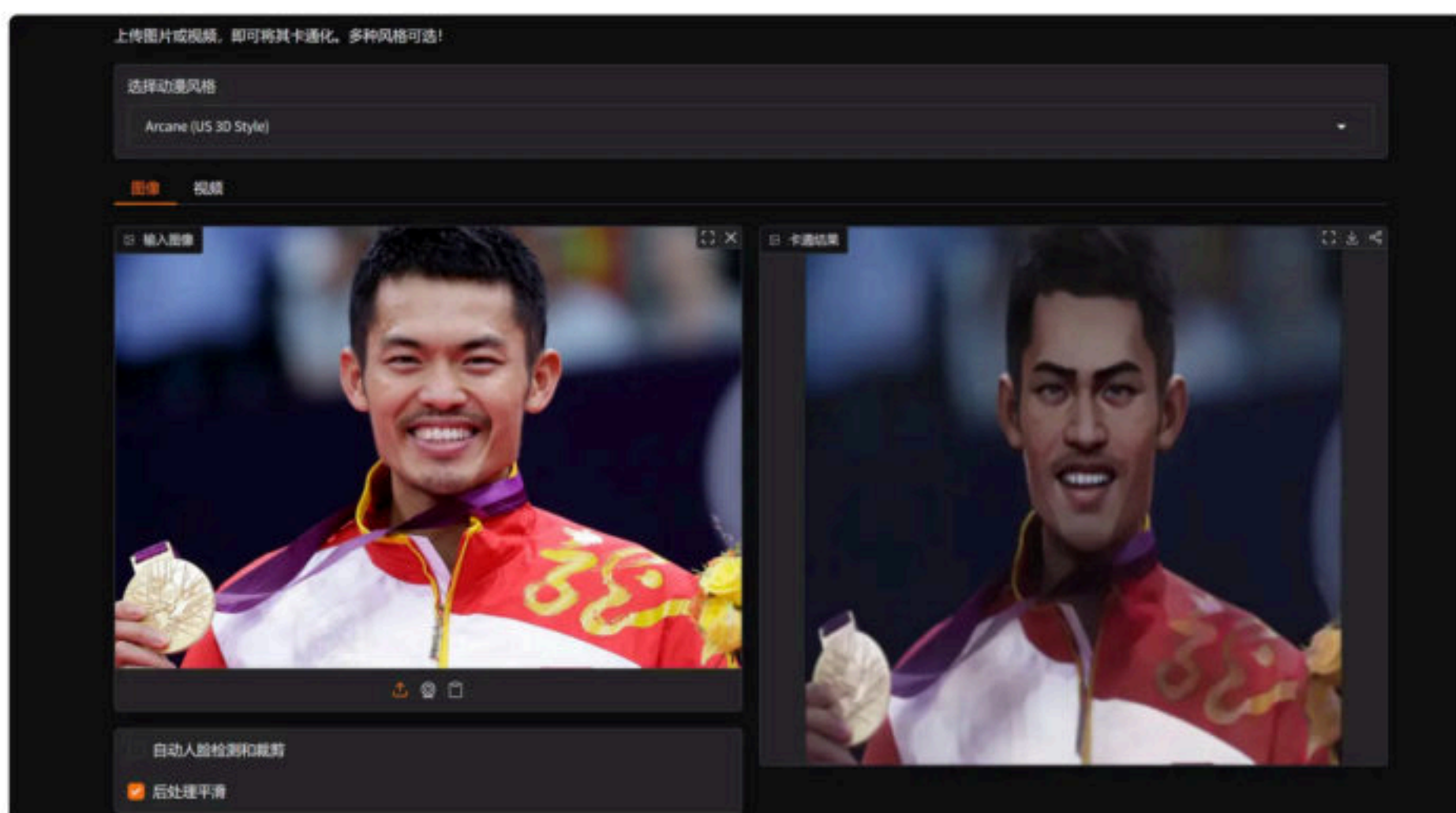
2. 模型权重来源说明 (Model Credits)

本项目所使用的预训练模型权重均来自开源社区的贡献，特此致谢：

- **AnimeGAN2-PyTorch**:
 - 作者: bryandlee
 - 仓库: <https://github.com/bryandlee/animegan2-pytorch>
 - 使用权重: `celeba_distill.pt` (宫崎骏风), `paprika.pt` (今敏风), `face_paint_512_v2.pt` (插画风)
- **ArcaneGAN**:

- 作者: Sxela (Alex Spirin)
- 仓库: <https://github.com/Sxela/ArcaneGAN>
- 使用权重: `ArcaneGANv0.4.jit` (双城之战风格)

3. 效果展示



CartoonGAN 风格迁移演示

上传图片或视频，即可将其卡通化。多种风格可选！

选择动漫风格

Arcane (US 3D Style)

